Gradient Boosting

Online Note on Gradient Boosting

AARHUS
UNIVERSITY

Boosting

- Meta Algorithm
  - Boosting takes any learning algorithm and builds a new one on top of it.
    - Base learning algorithm A.
    - Base hypothesis set H.

- Boosting:
  - Run A T times (not on same data set), producing hypotheses $h_1,\ldots,h_T \in H$.
  - Compute weights $\alpha_1,\ldots,\alpha_T$.
  - Return voting classifier:
    - $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$.



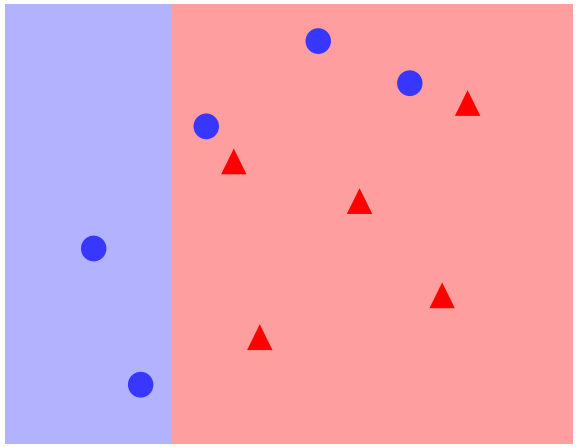
AdaBoost

- Input: Training data $(x_1,y_1),\ldots,(x_n,y_n)$. Base learning algorithm A.
- Initialize: $D_1(i)=1/n$ for all i.
- Algorithm:
  - For t=1,…,T:
    - Run A to minimize weighted 0-1 loss L_{D_t}. Outputs h_t.
    - $er_t = L_{D_t}(h_t) = \sum_{i=1}^{n} D_t(i)\, 1_{h_t(x_i)\neq y_i}$.
    - $\alpha_t = \frac{1}{2}\ln(\frac{1-er_t}{er_t})$.
    - $D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where

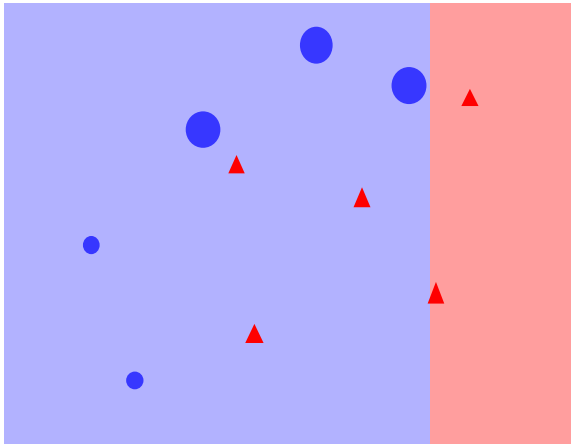
      $Z_t = \sum_{j=1}^{n} D_t(j)\exp(-\alpha_t y_j h_t(x_j))$.
- Output: $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$



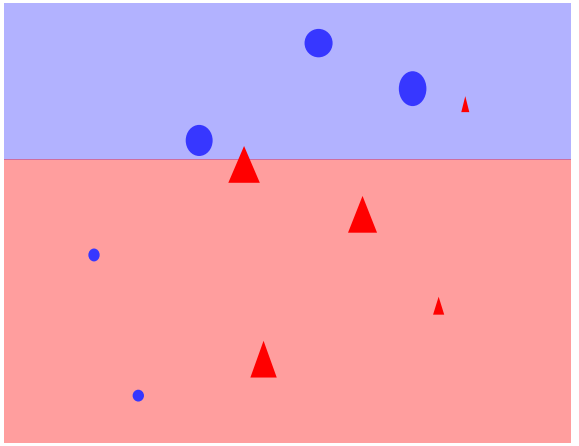
AdaBoost with Decision Stumps



$\alpha_1 \approx 0.42$

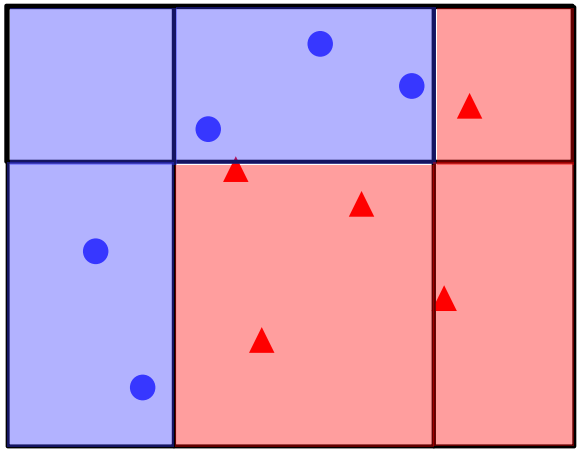


$\alpha_2 \approx 0.66$

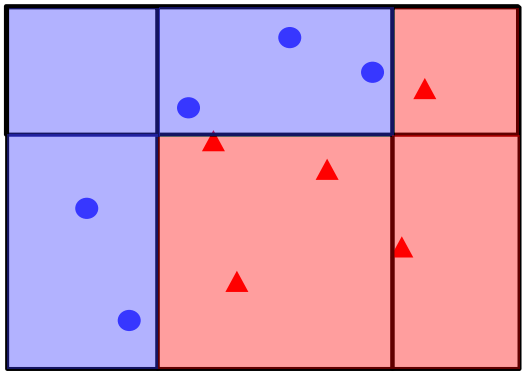


$\alpha_3 \approx 1.00$

$$f(x) = \text{sign}(0.42h_1(x) + 0.66h_2(x) + 1.00h_3(x))$$



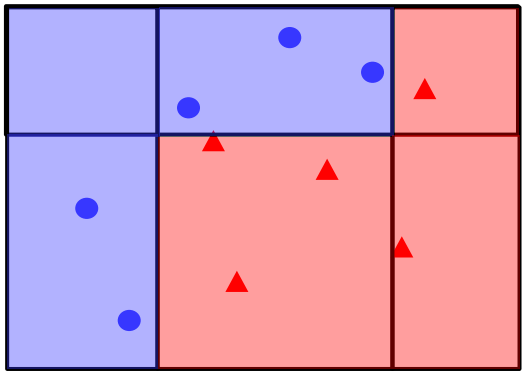
AdaBoost



- Issues
  - Minimizes exponential loss
    - $\frac{1}{n}\sum_{i=1}^{n} e^{-y_i f(x_i)}$
  - Not necessarily "desired" loss function.
  - Stated as binary classification:
    - $f(x) = \text{sign}(\sum_{t=1}^{T} \alpha_t h_t(x))$
  - What if want to do regression?
    - $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$ Least squares loss?
  - What if want sigmoid output like in logistic regression?
    - $f(x) = \theta(\sum_{t=1}^{T} \alpha_t h_t(x)) = \frac{1}{1+e^{-\sum_{t=1}^{T} \alpha_t h_t(x)}}$ Negative log-likelihood loss?



Gradient Boosting



- Gradient Boosting:
  - Alternative boosting method.
  - Allows arbitrary loss functions.
  - Uses regression trees as base learner.

- Motivating Example
  - Start simple
  - Regression:
    - $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
  - Least squares loss:
    - $E_{in}(f) = \frac{1}{n} \sum_{i=1}^{n} (f(x_i) - y_i)^2$

Gradient Boosting

- Regression:
  - $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
  - $E_{in}(f) = \frac{1}{n}\sum_{i=1}^{n}(f(x_i) - y_i)^2$
- Very natural algorithm:
  - Step 1:
    - Train regression tree h_1 to minimize least squares loss.

Example data:

(x_1,500)
(x_2,650)
(x_3,800)
(x_4,900)
(x_5,1150)

Example predictions:

$h_1(x_1)$=450
$h_1(x_2)$=500
$h_1(x_3)$=1000
$h_1(x_4)$=950
$h_1(x_5)$=1000

Mistakes y_i-$h_1(x_i)$:

500-450 =50
650-500 =150
800-1000 =-200
900-950 =-50
1150-1000 =150

Gradient Boosting

Example data:
(x_1,500)
(x_2,650)
(x_3,800)
(x_4,900)
(x_5,1150)

- Regression:
  - $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
  - $E_{in}(f) = \frac{1}{n}\sum_{i=1}^{n}(f(x_i) - y_i)^2$
- Very natural algorithm:
  - Step 1:
    - Train regression tree h_1 to minimize least squares loss.
  - Step 2 (boosting template):
    - Train h_2 and let $f(x) = h_1(x) + h_2(x)$
    - What would be a good h_2?
    - Train h_2 with data
      - (x_1, y_1-$h_1(x_1)$),…,(x_5, y_5-$h_1(x_5)$)
    - If perfect h_2 then
      - $f(x_i) = h_1(x_i) + \big(y_i - h_1(x_i)\big) = y_i$

Mistakes y_i-$h_1(x_i)$:

500-450	=50
650-500	=150
800-1000	=-200
900-950	=-50
1150-1000	=150

Gradient Boosting

Example data:
(x_1,500)
(x_2,650)
(x_3,800)
(x_4,900)
(x_5,1150)

- Regression:
  - $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
  - $E_{in}(f) = \frac{1}{n}\sum_{i=1}^{n}(f(x_i) - y_i)^2$
- Very natural algorithm:
  - Train regression tree h_1 to minimize least squares loss.
  - Train h_2 with data (x_1, y_1-$h_1(x_1)$),…,(x_5, y_5-$h_1(x_5)$).

Mistakes y_i-$h_1(x_i)$:		Example predictions:	Mistakes y_i-$f(x_i)$:	
500-450	=50	$h_2(x_1)$=80	500-450-80	=-30
650-500	=150	$h_2(x_2)$=120	650-500-120	=30
800-1000	=-200	$h_2(x_3)$=-200	800-1000+200	=0
900-950	=-50	$h_2(x_4)$=0	900-950-0	=-50
1150-1000	=150	$h_2(x_5)$=100	1150-1000-100	=50

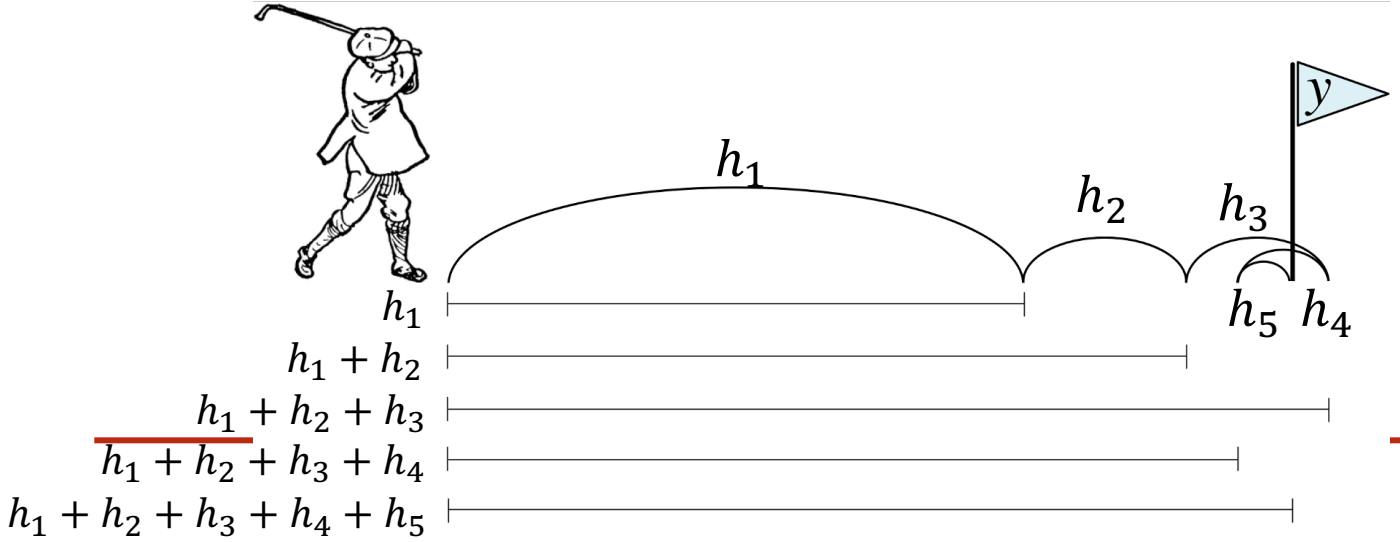
Gradient Boosting

- Regression:
  - $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
  - $E_{in}(f) = \frac{1}{n}\sum_{i=1}^{n}(f(x_i) - y_i)^2$
- Very natural algorithm:
  - Train regression tree h_1 to minimize least squares loss.
  - Train h_2 with data $(x_1, y_1\text{-}h_1(x_1)),\ldots,(x_5, y_5\text{-}h_1(x_5))$.
  - Train h_3 with data $(x_1, y_1\text{-}h_1(x_1)\text{-}h_2(x_1)),\ldots,(x_5, y_5\text{-}h_1(x_5)\text{-}h_2(x_5))$.
  - …
  - Train h_T with data $(x_1, y_1\text{-}h_1(x_1)\text{-}h_2(x_1)\text{-}\ldots\text{-}h_{T-1}(x_1)),\ldots,(x_5, y_5\text{-}h_1(x_5)\text{-}h_2(x_5)\text{-}\ldots\text{-}h_{T-1}(x_5))$.

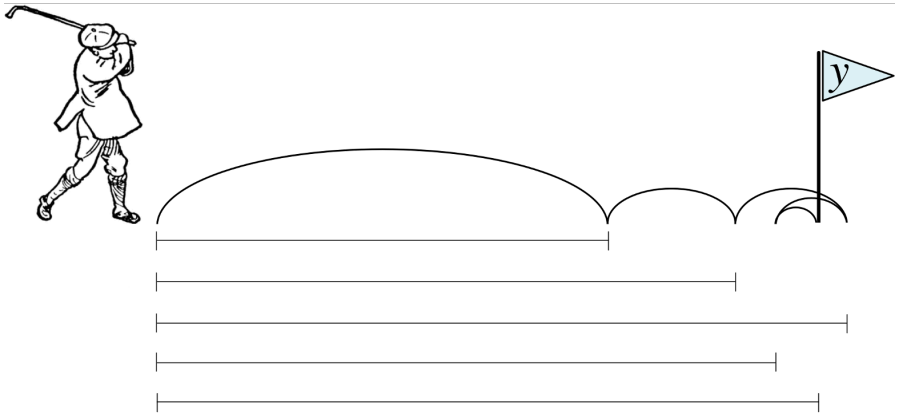


Gradient Boosting

- Regression:
  - $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
  - $E_{in}(f) = \frac{1}{n} \sum_{i=1}^{n} (f(x_i) - y_i)^2$
- Very natural algorithm:
  - Train regression tree h_1 to minimize least squares loss.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data $(x_i, y_i - \sum_{j=1}^{t-1} h_j(x_i))$.



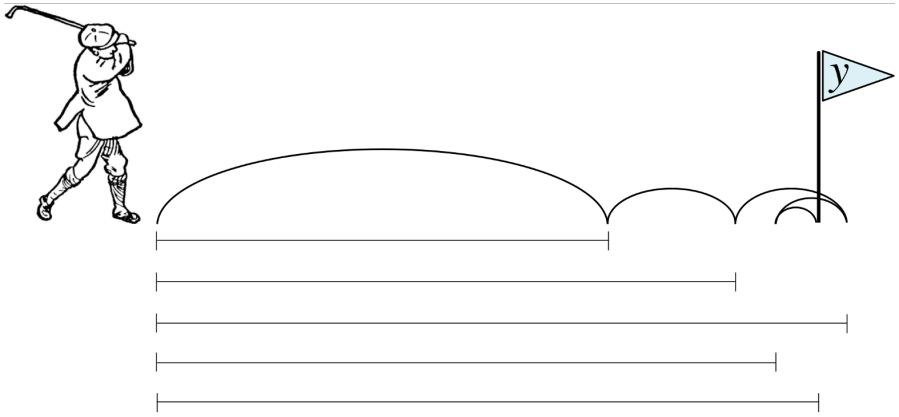
Gradient Boosting



- Very natural algorithm:
  - Train regression tree h_1 to minimize least squares loss.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data $(x_i, y_i - \sum_{j=1}^{t-1} h_j(x_i))$.

- Extra Tweek:
  - Risk of "overstepping" target when add h_t.
  - Introduce learning rate η.
    - Add h_t to f with $\alpha_t = \eta$

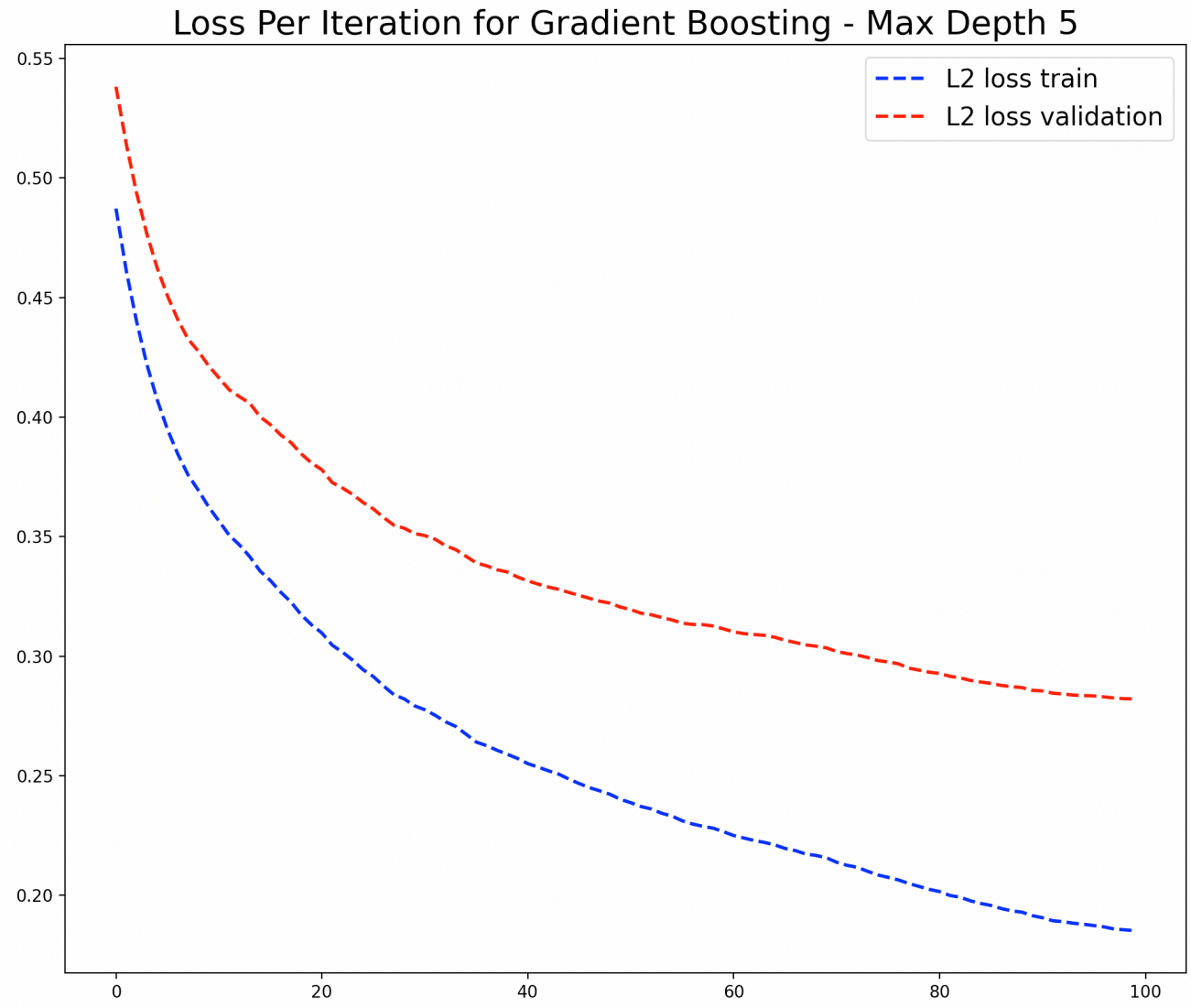
Gradient Boosting



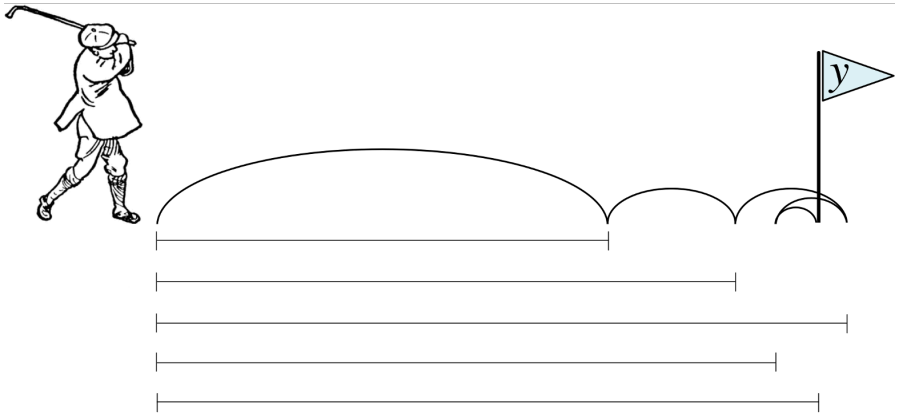
- Very natural algorithm:
  - Train regression tree h_1 to minimize least squares loss.
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data $(x_i, y_i - \sum_{j=1}^{t-1} \alpha_j h_j(x_i))$.
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

Gradient Boosting

- Experiment:
  - California house prices.
  - Learning rate 0.1
- Much than just one regression tree.



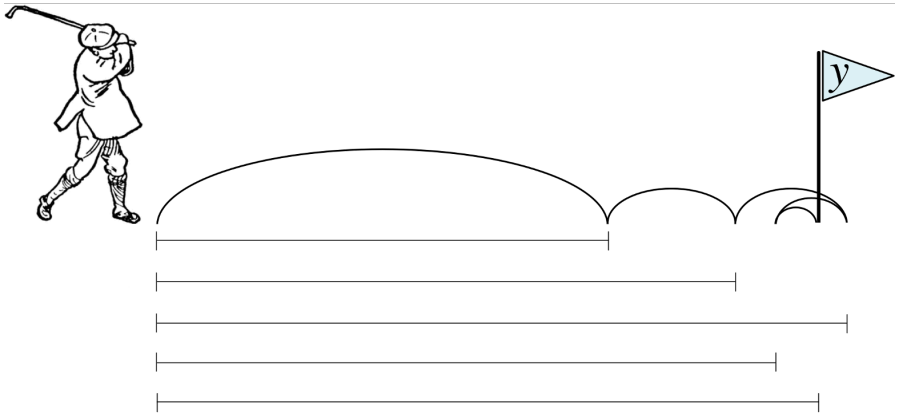
Gradient Boosting



- Very natural algorithm:
  - Train regression tree h_1 to minimize least squares loss.
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data $(x_i, y_i - \sum_{j=1}^{t-1} \alpha_j h_j(x_i))$.
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

- Small modification:
  - To prepare for ”full” gradient boosting algorithm.
  - Replace h_1 by best constant function $h_1(\mathrm{x}) = \mathrm{c}$.

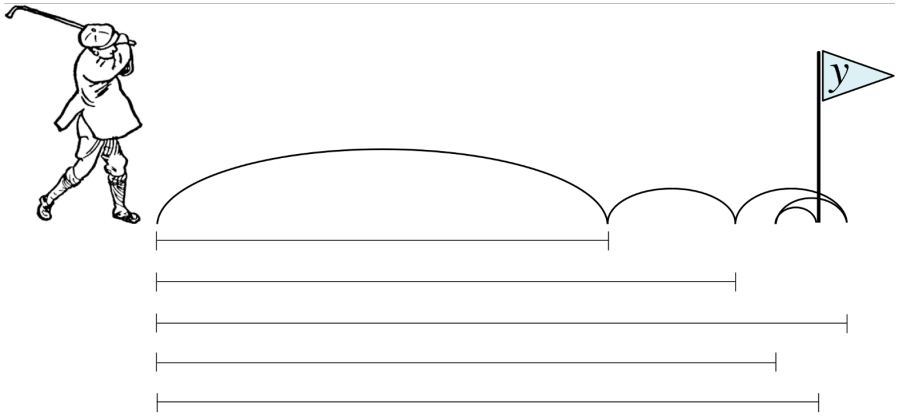
Gradient Boosting



- Very natural algorithm:
  - Let $h_1(x) = c$ minimize least squares loss.
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data $(x_i, y_i - \sum_{j=1}^{t-1} \alpha_j h_j(x_i))$.
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

- Small modification:
  - To prepare for ”full” gradient boosting algorithm.
  - Replace h_1 by best constant function $h_1(\mathrm{x}) = \mathrm{c}$.

Gradient Boosting



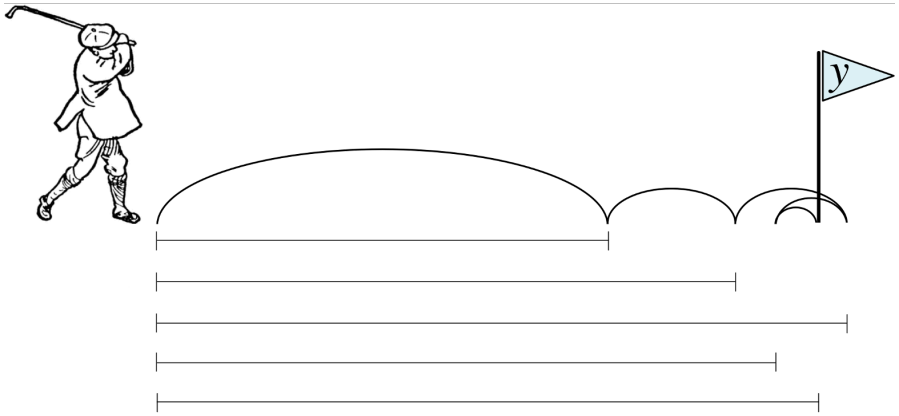
- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data $(x_i, y_i - \sum_{j=1}^{t-1} \alpha_j h_j(x_i))$.
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

Mean minizer for least squares (shown in exercise earlier)

- Small modification:
  - To prepare for ”full” gradient boosting algorithm.
  - Replace h_1 by best constant function $h_1(\mathrm{x}) = \mathrm{c}$.



Gradient Boosting



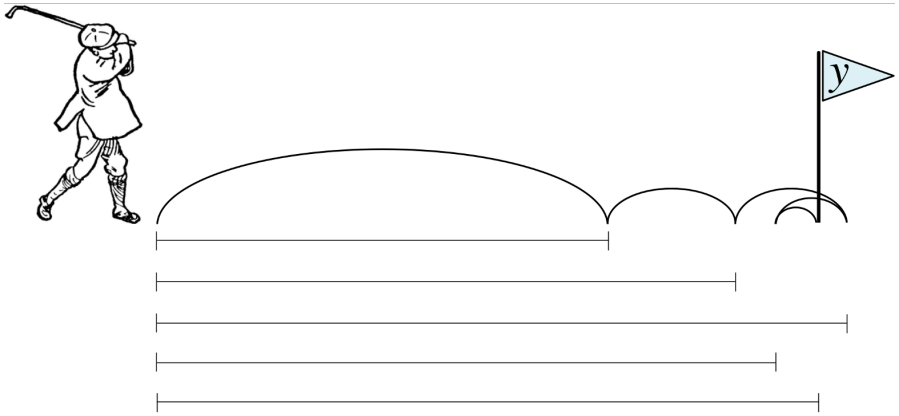
- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data $(x_i, y_i - \sum_{j=1}^{t-1} \alpha_j h_j(x_i))$.
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

Gradient Boosting

Online Note on Gradient Boosting

AARHUS
UNIVERSITY

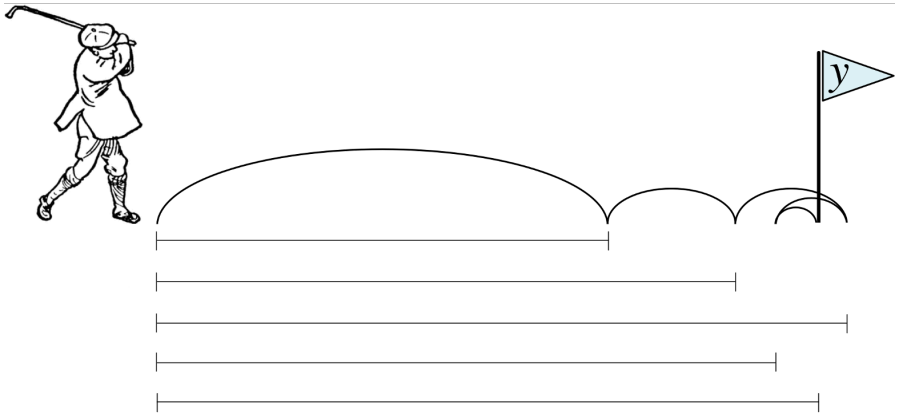
Gradient Boosting



- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data $(x_i, y_i - \sum_{j=1}^{t-1} \alpha_j h_j(x_i))$.
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
- Towards final algorithm:
  - Introduce notation $F_t(x) = \sum_{j=1}^{t} \alpha_j h_j(x)$
  - For data matrix X, let $F_t(X) = \begin{pmatrix} \sum_{j=1}^{t} \alpha_j h_j(x_1) \\ \vdots \\ \sum_{j=1}^{t} \alpha_j h_j(x_n) \end{pmatrix}$



Gradient Boosting



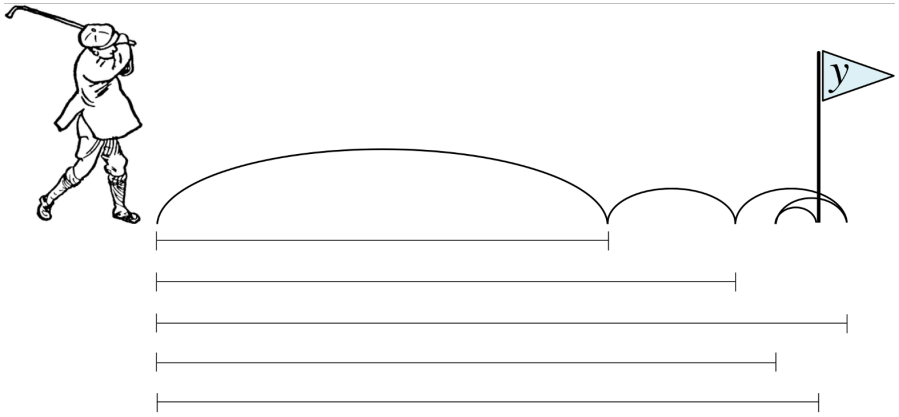
$F_t(x) = \sum_{j=1}^{t} \alpha_j h_j(x)$.

- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = y - F_{t-1}(X)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

- Observation: If perfect h_t then
  - $F_t(X) = F_{t-1}(X) + \eta\hat{y}$

Looks like another algorithm we have seen!

Gradient Boosting



$F_t(x) = \sum_{j=1}^{t} \alpha_j h_j(x)$.

- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = y - F_{t-1}(X)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
- Observation: If perfect h_t then
  - $F_t(X) = F_{t-1}(X) + \eta\hat{y}$

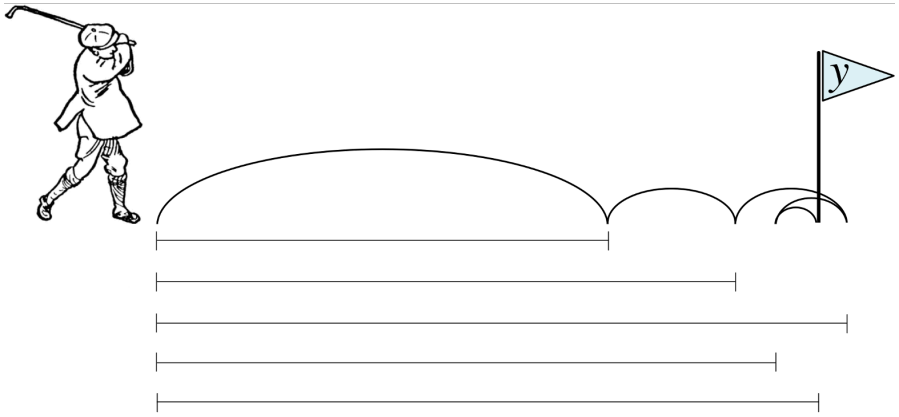
Gradient Descent on predictions F_{t-1}:

$$E_{in}(F_{t-1}(X)) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - F_{t-1}(x_i)\right)^2.$$

$$\nabla_{F_{t-1}(X)} E_{in}(F_{t-1}) = \begin{pmatrix} \frac{\partial E_{in}(F_{t-1})}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial E_{in}(F_{t-1})}{\partial F_{t-1}(x_n)} \end{pmatrix}$$



Gradient Boosting



$F_t(x) = \sum_{j=1}^{t} \alpha_j h_j(x)$.

- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = y - F_{t-1}(X)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

- Observation: If perfect h_t then
  - $F_t(X) = F_{t-1}(X) + \eta\hat{y}$

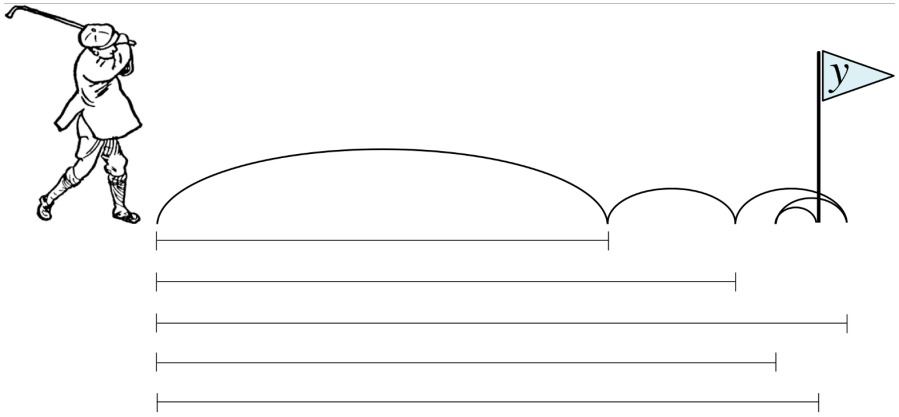
Gradient Descent on predictions F_{t-1}:

$$E_{in}(F_{t-1}(X)) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - F_{t-1}(x_i)\right)^2.$$

$$\nabla_{F_{t-1}(X)} E_{in}(F_{t-1}) = \begin{pmatrix} -\frac{2}{n}(y_1 - F_{t-1}(x_1)) \\ \vdots \\ -\frac{2}{n}(y_n - F_{t-1}(x_n)) \end{pmatrix}$$



Gradient Boosting



$F_t(x) = \sum_{j=1}^{t} \alpha_j h_j(x)$.

- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = y - F_{t-1}(X)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

Gradient Descent on predictions F_{t-1}:

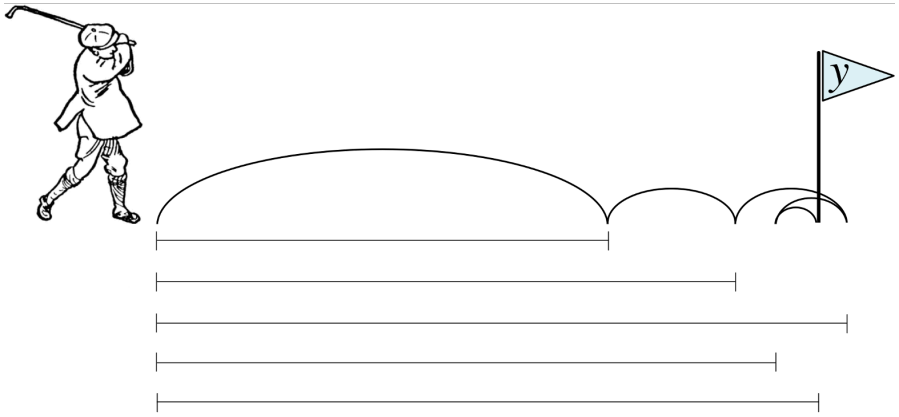
$$E_{in}(F_{t-1}(X)) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - F_{t-1}(x_i)\right)^2.$$

$$\nabla_{F_{t-1}(X)} E_{in}(F_{t-1}(X)) = -\frac{2}{n}\hat{y}$$

- Observation: If perfect h_t then
  - $F_t(X) = F_{t-1}(X) + \eta\hat{y}$



Gradient Boosting



$F_t(x) = \sum_{j=1}^{t} \alpha_j h_j(x)$.

- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = y - F_{t-1}(X)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

- Observation: If perfect h_t then
  - $F_t(X) = F_{t-1}(X) + \eta\hat{y}$

Gradient Descent on predictions F_{t-1}:

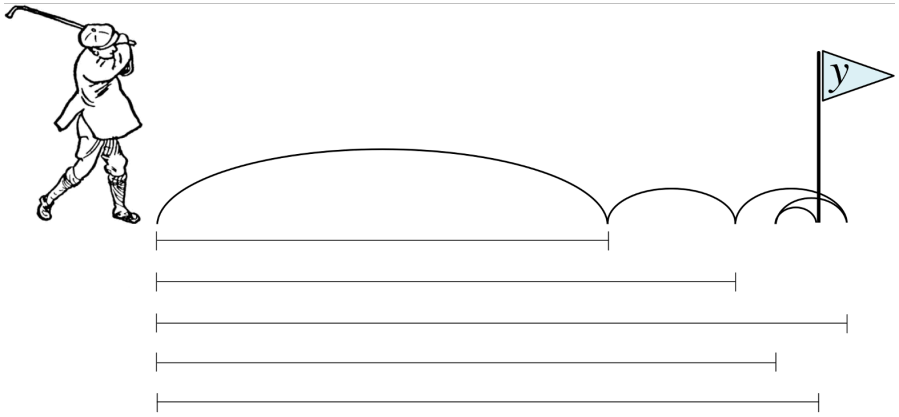
$$E_{in}(F_{t-1}(X)) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - F_{t-1}(x_i)\right)^2.$$

$$\nabla_{F_{t-1}(X)} E_{in}(F_{t-1}(X)) = -\frac{2}{n}\hat{y}$$

$$F_t(X) = F_{t-1}(X) - \left(-\eta\frac{2}{n}\right)\hat{y}$$



Gradient Boosting



$F_t(x) = \sum_{j=1}^{t} \alpha_j h_j(x)$.

- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = y - F_{t-1}(X)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

- Observation: If perfect h_t then
  - $F_t(X) = F_{t-1}(X) + \eta\hat{y}$

Gradient Descent on predictions F_{t-1}:

$$E_{in}(F_{t-1}(X)) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - F_{t-1}(x_i)\right)^2.$$

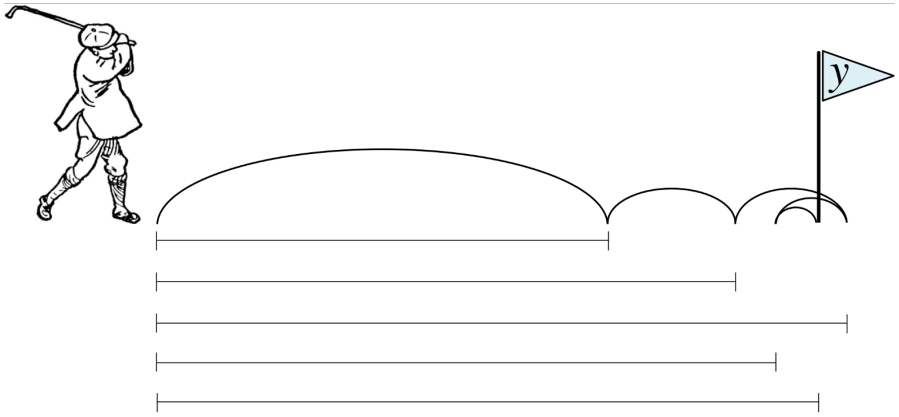
$$\nabla_{F_{t-1}(X)} E_{in}(F_{t-1}(X)) = -\frac{2}{n}\hat{y}$$

$$F_t(X) = F_{t-1}(X) + \eta'\hat{y}$$

Same Algorithm up to scaling of learning rate!



Gradient Boosting



$F_t(x) = \sum_{j=1}^{t} \alpha_j h_j(x)$.

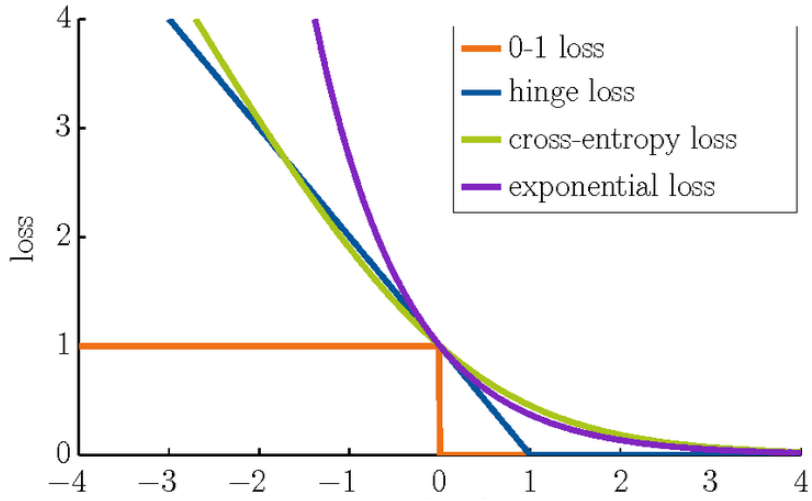
- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$
  - Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = y - F_{t-1}(X)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

- Observation: If perfect h_t then:
  - Performs gradient descent on predictions to minimize E_{in}.
  - This is the key idea for generalizing to other loss functions!



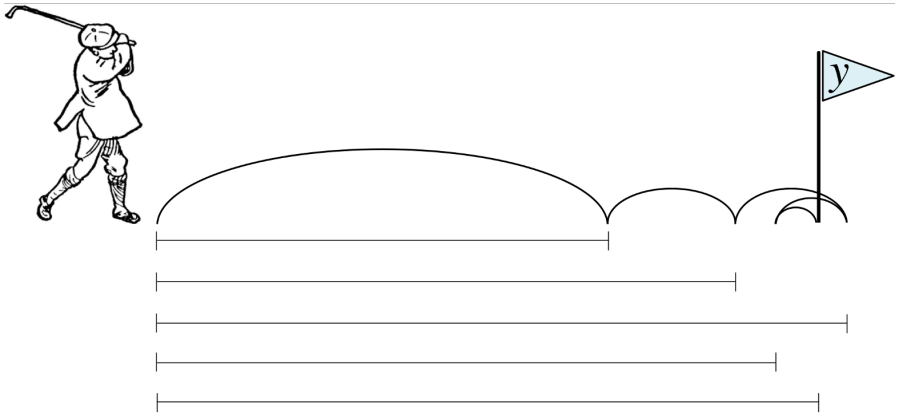
Pointwise Loss

- Any pointwise loss function:
  - $E_{in}(f) = \frac{1}{n}\sum_{i=1}^{n} L(f(x_i), y_i)$



- Assume L is differentiable and can compute derivative wrt. $f(x)$:
  - Examples:
  - $L(f(x), y) = \big(y - f(x)\big)^2, \frac{\partial L(f(x),y)}{\partial f(x)} = -2(y - f(x))$ Least squares
  - $L(f(x), y) = -\ln(\theta(yf(x)), \frac{\partial L(f(x),y)}{\partial f(x)} = y(\theta\big(yf(x)\big) - 1)$ NLL(sigmoid) = cross entropy
  - $L(f(x), y) = \mathrm{e}^{-yf(x)}, \frac{\partial L(f(x),y)}{\partial f(x)} = -ye^{-yf(x)}$ exponential
  - $L(f(x), y) = \max\big(0, 1 - yf(x)\big), \frac{\partial L(f(x),y)}{\partial f(x)} = -y1_{1-yf(x)>0}$ hinge

Gradient Boosting



- Very natural algorithm:
  - Let $h_1(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = y - F_{t-1}(X)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

$F_t(x) = \sum_{j=1}^{t} \alpha_j h_j(x)$.

Gradient Boosting

- Towards Gradient Boosting:
  - Let $h_1(x) = c$ where c minimizes $\mathrm{E}_{\mathrm{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = -\nabla_{F_{t-1}(X)} E_{in}(F_{t-1}(X)) = -\frac{1}{n}\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$.
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
- Observation: If perfect h_t then:
  - Performs gradient descent on predictions to minimize E_{in}.
    - $F_t(X) = F_{t-1}(X) - \eta \nabla_{F_{t-1}(X)} E_{in}(F_{t-1}(X))$



Gradient Boosting

- Towards Gradient Boosting:
  - Let $h_1(x) = c$ where c minimizes $\mathrm{E}_{\mathrm{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = -\nabla_{F_{t-1}(X)} E_{in}(F_{t-1}(X)) = -\frac{1}{n}\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$.
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
- Observation:
  - 1/n factor in E_{in} results in very small changes in each prediction. Drop it - still gradient descent with different η.



Gradient Boosting

- Towards Gradient Boosting:
  - Let $h_1(x) = c$ where c minimizes $\mathrm{E}_{\mathrm{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = -\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$. Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$



Gradient Boosting

- Towards Gradient Boosting:
  - Let $h_1(x) = c$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}$ $\sum_{i=1} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss

      with data X, $\hat{y} = -\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$ Let $\alpha_t =$
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t$

Positive: If perfect, then perfect gradient descent.

Downside: Minimizing wrong loss.

Positive: Don't need custom-made tree learning algorithm.



Gradient Boosting

- Towards Gradient Boosting:
  - Let $h_1(x) = c$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = -\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$. Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
- One last optimization:
  - Nice to use same tree building algorithm (least squares).
  - Simple optimization: Change values returned in leaves.



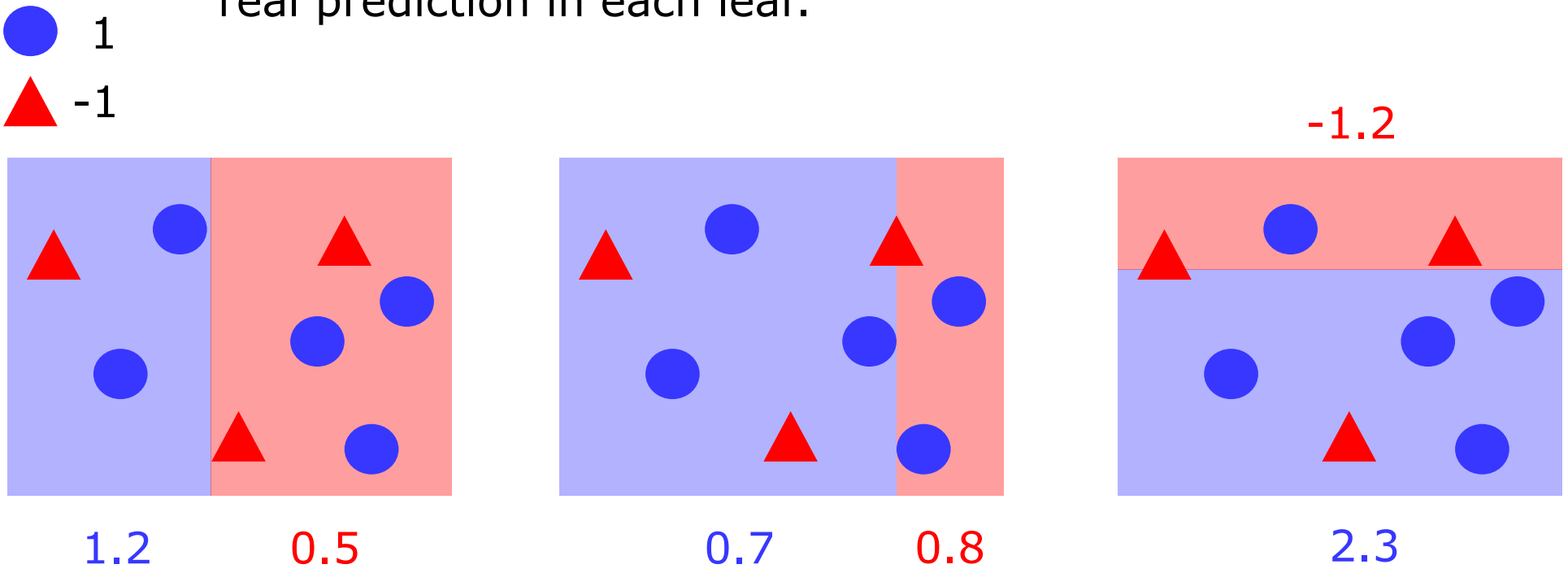
Gradient Boosting

- Gradient Boosting:
  - Let $h_1(x) = c$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = -\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$.
    - For each leaf l in h_t:
      - Let l predict value c_l minimizing
        - $\sum_{i \in l} L(F_{t-1}(x_i) + c_l, y_i)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$



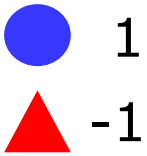
Gradient Boosting with Regression Stumps

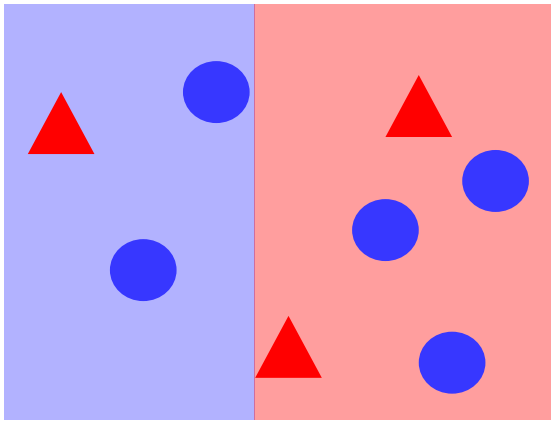
- Regression Stumps
  - Hypothesis set: All axis-parallel hyperplanes/lines, with a real prediction in each leaf.

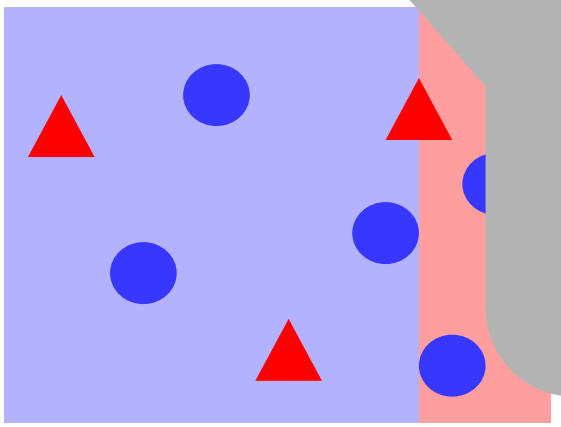


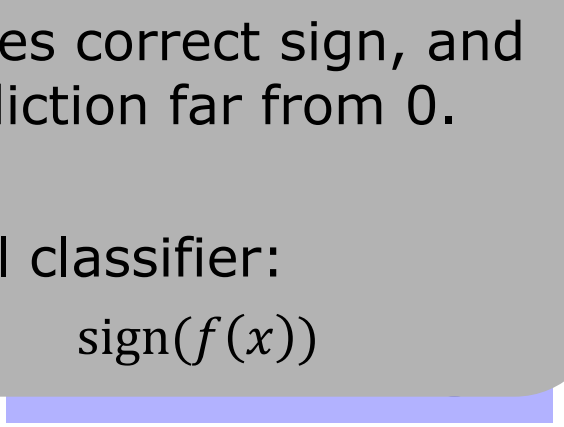
Gradient Boosting with Regression Stumps with Hinge Loss

- Regression Stumps
  - Hypothesis set: All axis-parallel hyperplanes/lines, with a real prediction in each leaf.
  - $L(f(x), y) = \max(0, 1 - yf(x))$









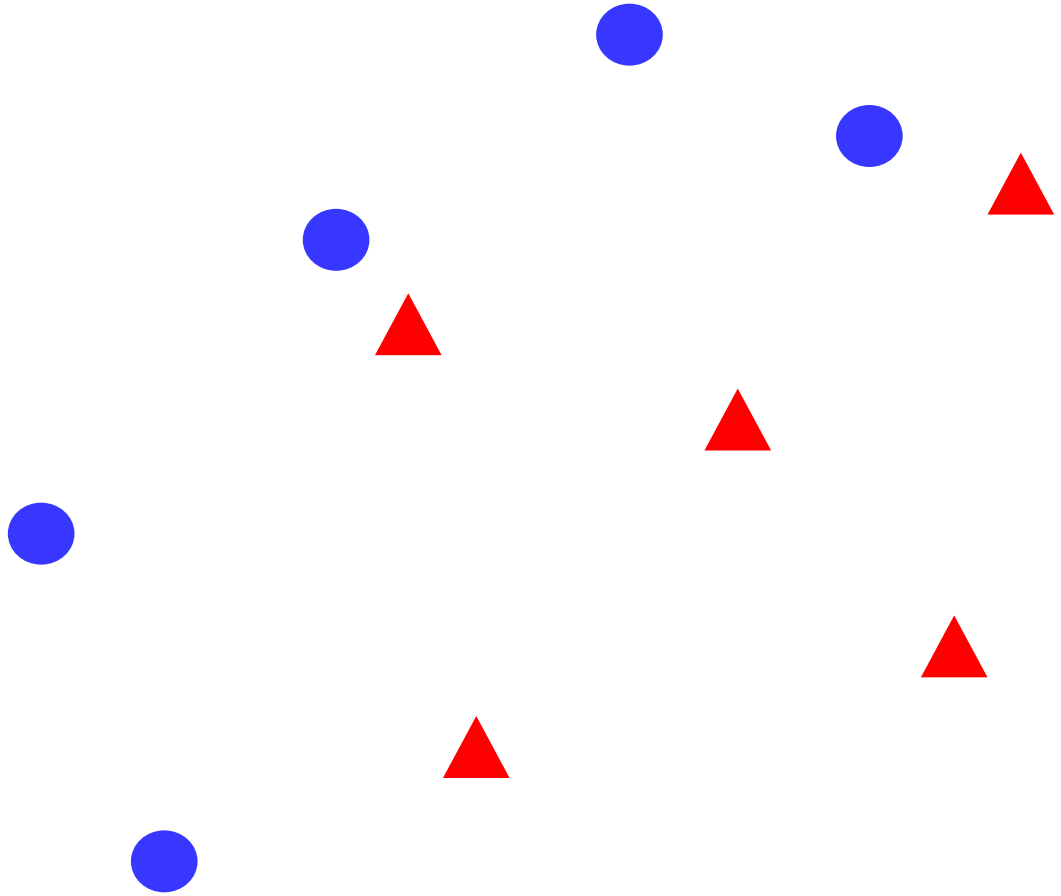
1.2 0.5 0.7 0.8 2.3

0 if $yf(x) \geq 1$.
Forces correct sign, and prediction far from 0.

Final classifier:

$$\text{sign}(f(x))$$

Gradient Boosting, Stumps, Hinge

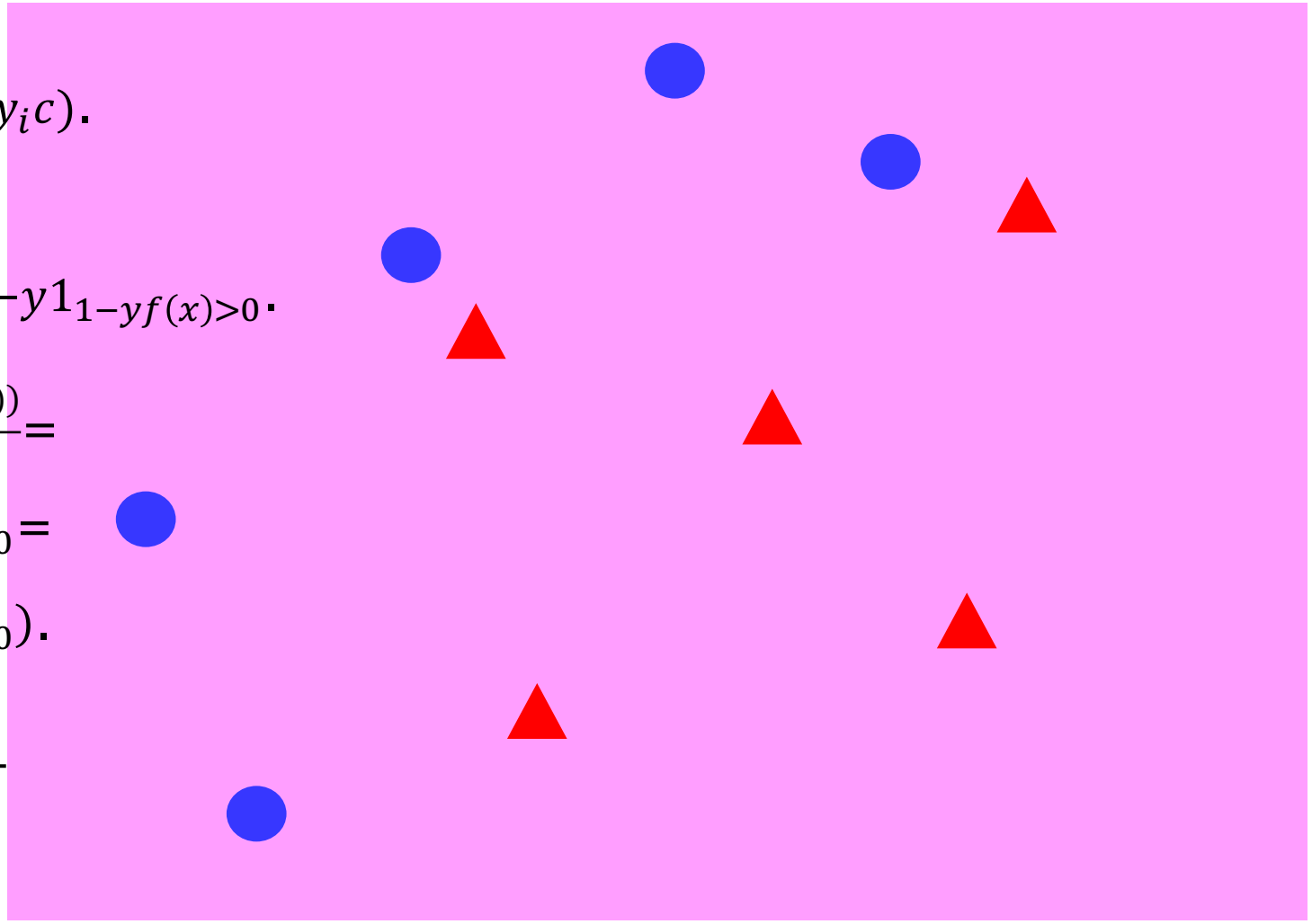


Gradient Boosting

- Gradient Boosting:
  - Let $h_1(x) = c$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = -\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$.
    - For each leaf l in h_t:
      - Let l predict value c_l minimizing
        - $\sum_{i \in l} L(F_{t-1}(x_i) + c_l, y_i)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$



Gradient Boosting, Stumps, Hinge



$\frac{1}{n}\sum_{i=1}^{n}\max(0,1-y_i c)$.

Derivative:

$\frac{\partial \max(0,1-yf(x))}{\partial f(x)} = -y1_{1-yf(x)>0}$.

$\frac{\partial(\frac{1}{n}\sum_{i=1}^{n}\max(0,1-y_i c))}{\partial c} =$

$\frac{1}{n}\sum_{i=1}^{n} -y_i 1_{1-y_i c>0} =$

$\frac{1}{2}(1_{1+c>0} - 1_{1-c>0})$.

0 when -1<c<1

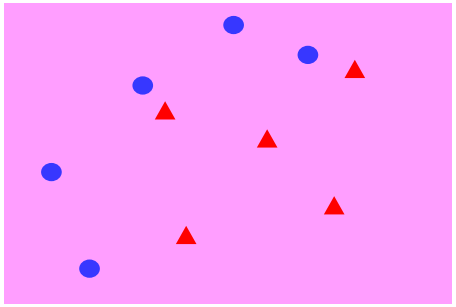
$-$ when $c \leq -1$

$+$ when $c \geq 1$

Can choose c=0

0

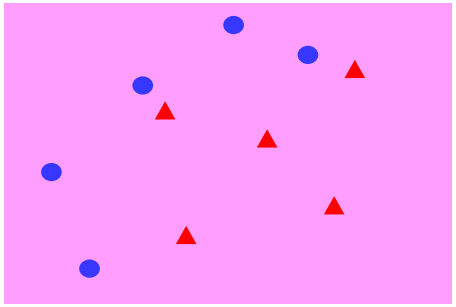
Gradient Boosting



- Gradient Boosting:
  - Let $h_1(x) = 0$ where c minimizes $\mathrm{E}_{\mathrm{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = -\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$.
    - For each leaf l in h_t:
      - Let l predict value c_l minimizing
        - $\sum_{i \in l} L(F_{t-1}(x_i) + c_l, y_i)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$



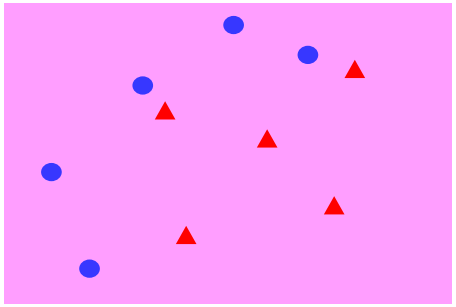
Gradient Boosting



- Gradient Boosting:
  - Let $h_1(x) = 0$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = -\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$.
    - For each leaf l in h_t:
      - Let l predict value c_l minimizing
        - $\sum_{i \in l} L(F_{t-1}(x_i) + c_l, y_i)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$



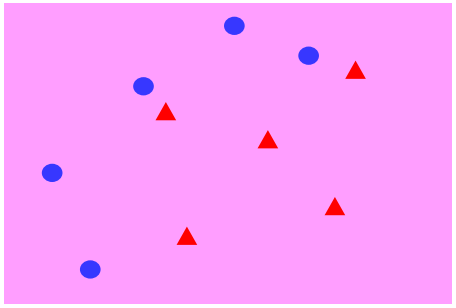
Gradient Boosting



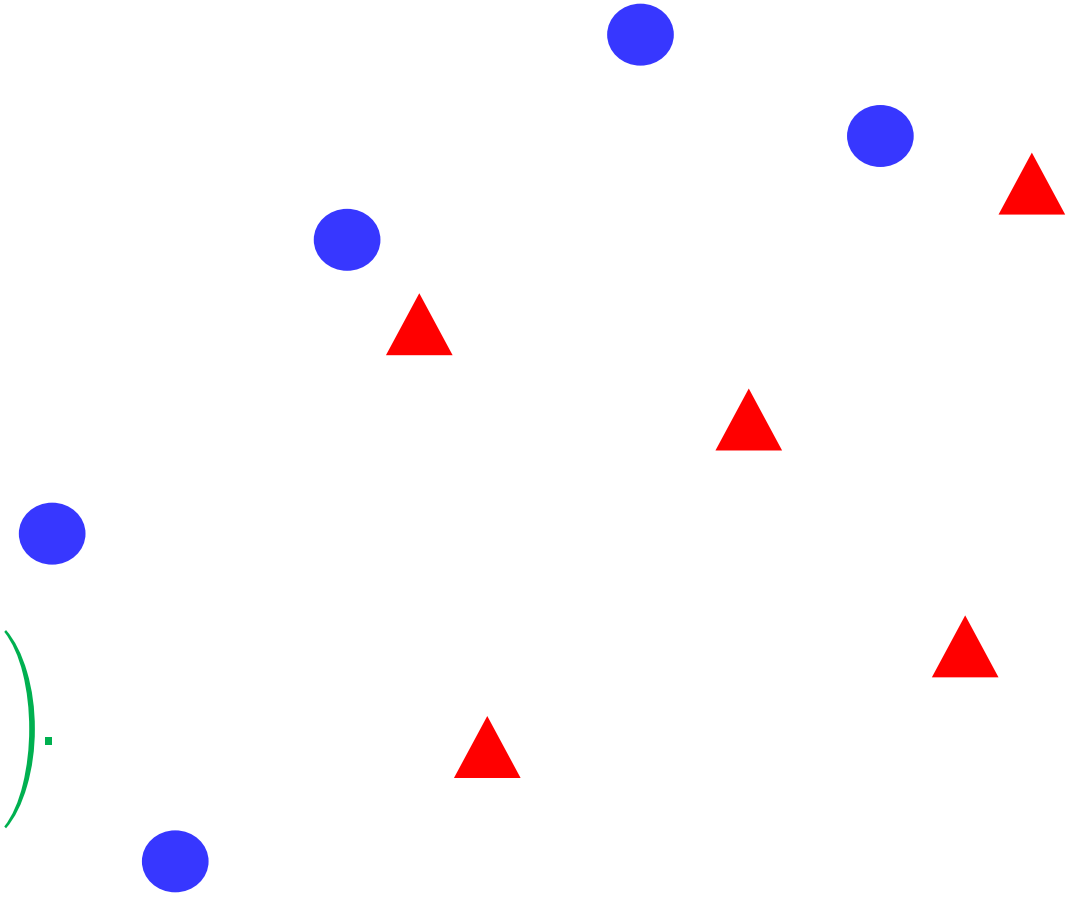
- Gradient Boosting:
  - Let $h_1(x) = 0$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = \begin{pmatrix} y_1 1_{1-y_1 F_{t-1}(x_1)>0} \\ \vdots \\ y_n 1_{1-y_n F_{t-1}(x_n)>0} \end{pmatrix}$. Hinge Loss
    - For each leaf l in h_t:
      - Let l predict value c_l minimizing
        - $\sum_{i \in l} L(F_{t-1}(x_i) + c_l, y_i)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$



Gradient Boosting, Stumps, Hinge



0



1

-1

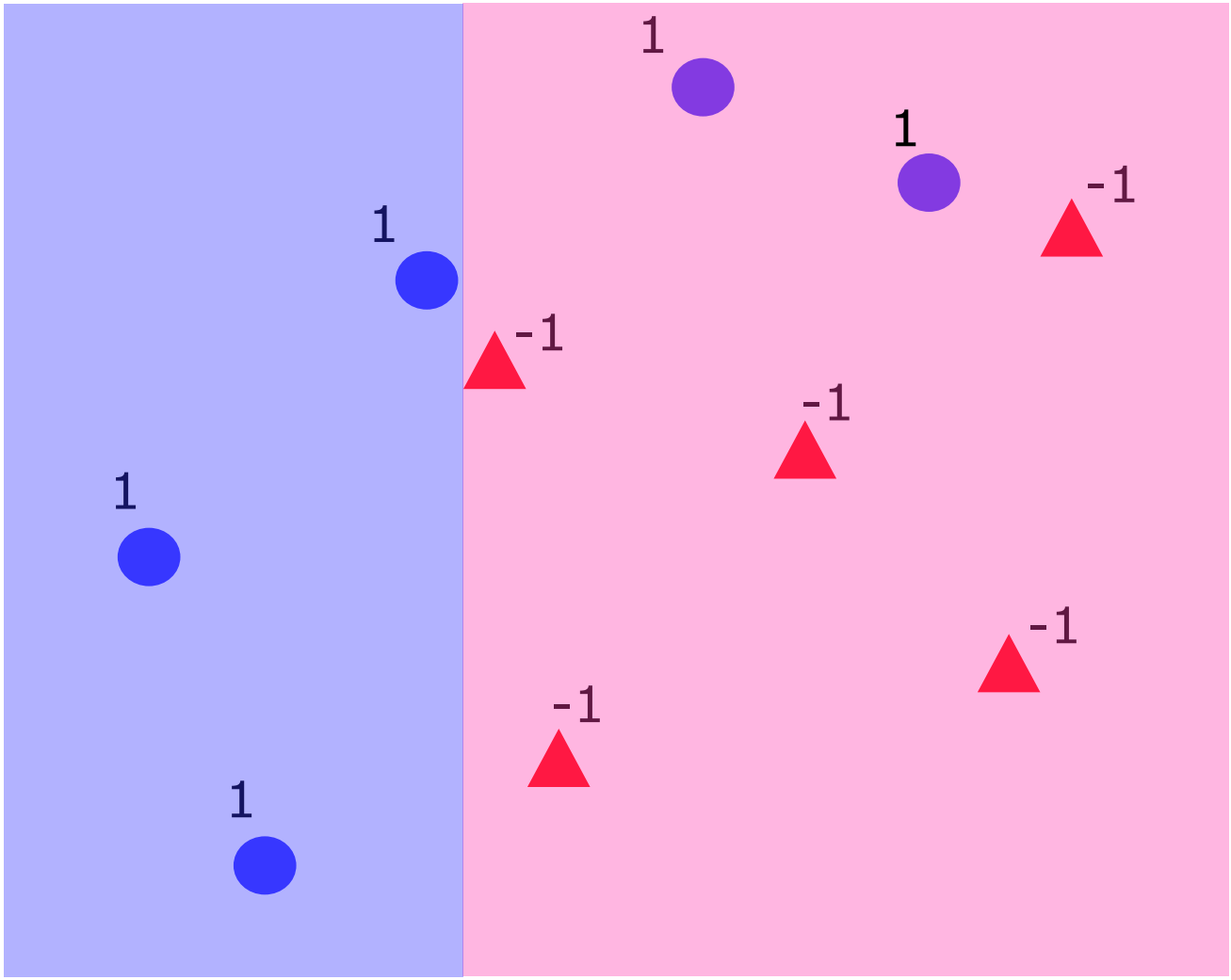
$$\hat{y} = \begin{pmatrix} y_1 1_{1-y_1 F_{t-1}(x_1)>0} \\ \vdots \\ y_n 1_{1-y_n F_{t-1}(x_n)>0} \end{pmatrix}.$$

$$\hat{y} = \begin{pmatrix} y_1 1_{1>0} \\ \vdots \\ y_n 1_{1>0} \end{pmatrix} = y.$$

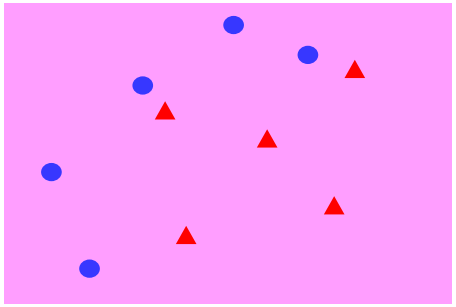


Gradient Boosting, Stumps, Hinge

Try all splits and choose one with smallest least squares when return mean in leaves



Gradient Boosting

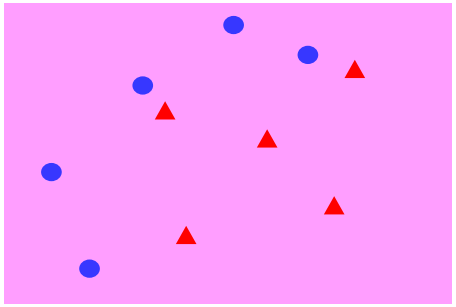


- Gradient Boosting:
  - Let $h_1(x) = 0$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = \begin{pmatrix} y_1 1_{1-y_1 F_{t-1}(x_1)>0} \\ \vdots \\ y_n 1_{1-y_n F_{t-1}(x_n)>0} \end{pmatrix}$.
    - For each leaf l in h_t:
      - Let l predict value c_l minimizing
        - $\sum_{i \in l} L(F_{t-1}(x_i) + c_l, y_i)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

Hinge Loss



Gradient Boosting



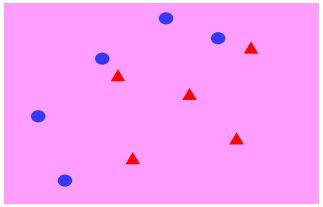
- Gradient Boosting:
  - Let $h_1(x) = 0$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = \begin{pmatrix} y_1 1_{1-y_1 F_{t-1}(x_1)>0} \\ \vdots \\ y_n 1_{1-y_n F_{t-1}(x_n)>0} \end{pmatrix}$.
    - For each leaf l in h_t:
      - Let l predict value c_l minimizing
        - $\sum_{i \in l} \max(0, 1 - y_i(F_{t-1}(x_i) + c_l))$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

0

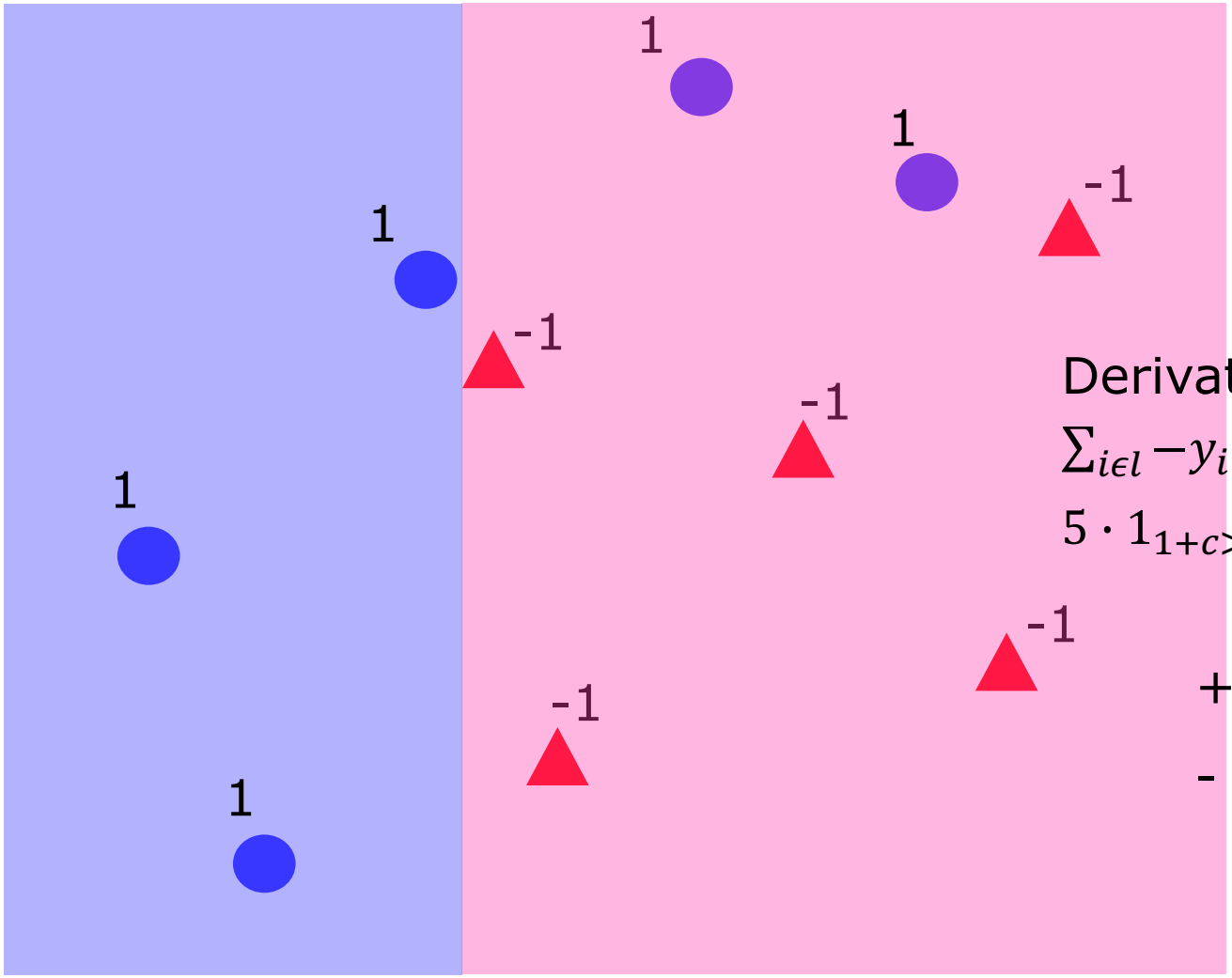
Hinge Loss



Gradient Boosting, Stumps, Hinge



0



Mean
=-3/7

Derivative:

$\sum_{i\epsilon l} -y_i 1_{1-y_i c>0} =$

$5 \cdot 1_{1+c>0} - 2 \cdot 1_{1-c>0}.$

\+ when c>-1

\- when c<-1

-1

1

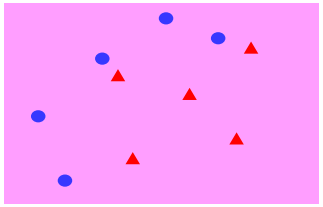
Mean
=1

minimize

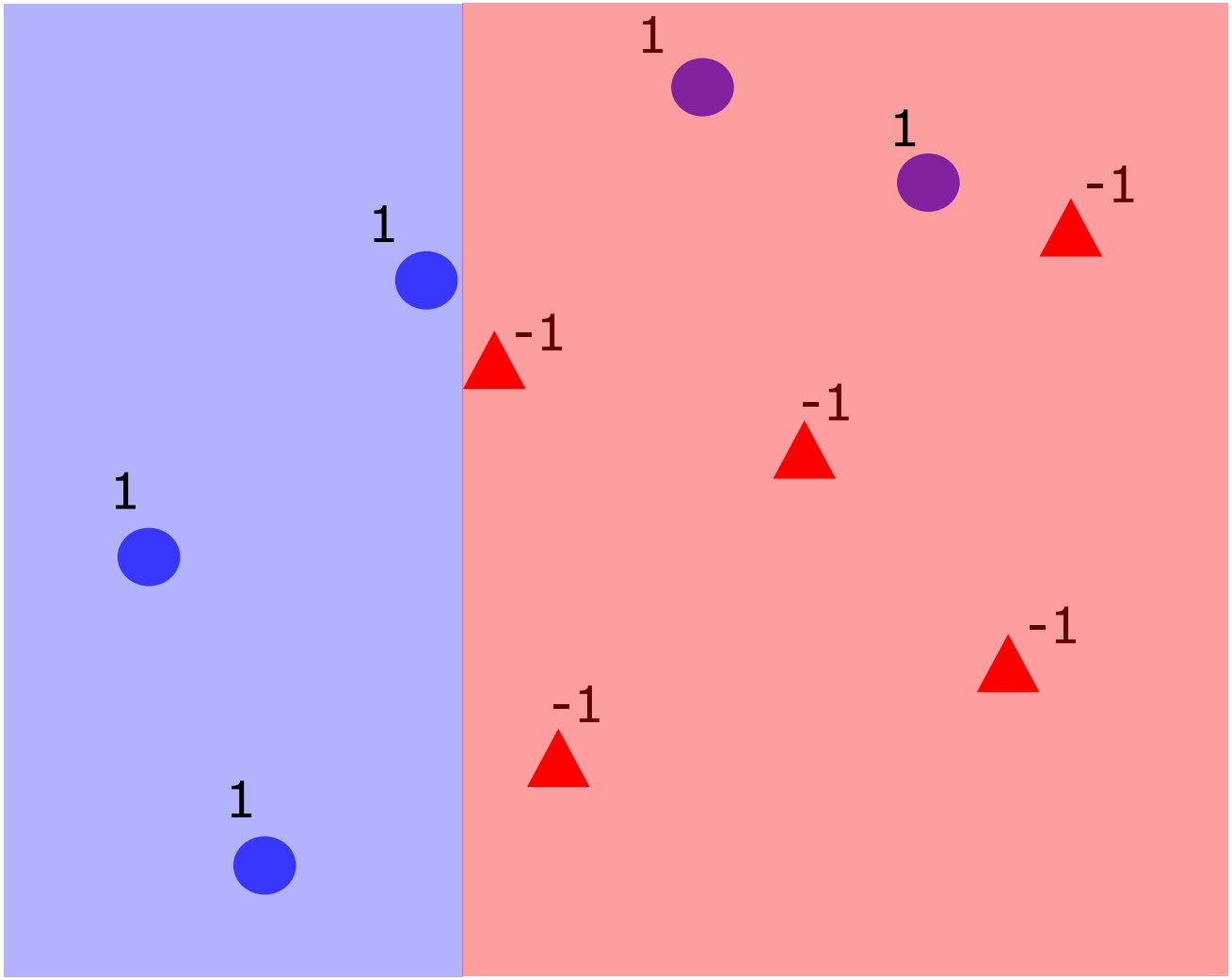
$\sum_{i\in l} \max(0, 1 - y_i(F_{t-1}(x_i) + c_l)) = \sum_{i\in l} \max(0, 1 - y_i c_l).$



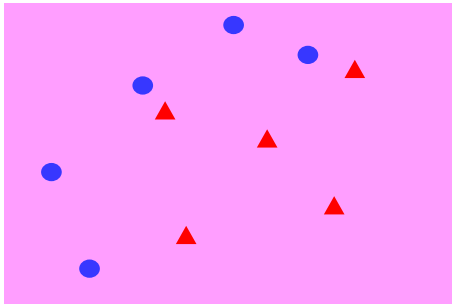
Gradient Boosting, Stumps, Hinge



0



Gradient Boosting



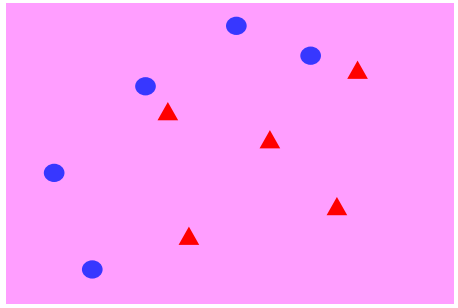
- Gradient Boosting:
  - Let $h_1(x) = 0$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = \begin{pmatrix} y_1 1_{1-y_1 F_{t-1}(x_1)>0} \\ \vdots \\ y_n 1_{1-y_n F_{t-1}(x_n)>0} \end{pmatrix}$.
    - For each leaf l in h_t:
      - Let l predict value c_l minimizing
        - $\sum_{i \in l} \max(0, 1 - y_i(F_{t-1}(x_i) + c_l))$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

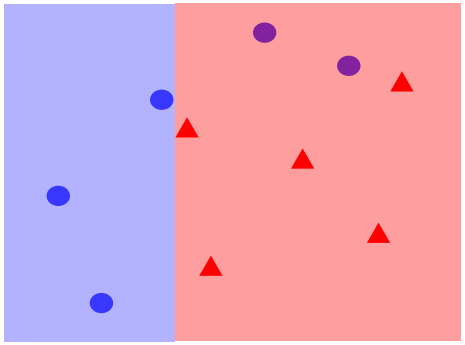
0

Hinge Loss



Gradient Boosting, Stumps, Hinge





0

1

-1

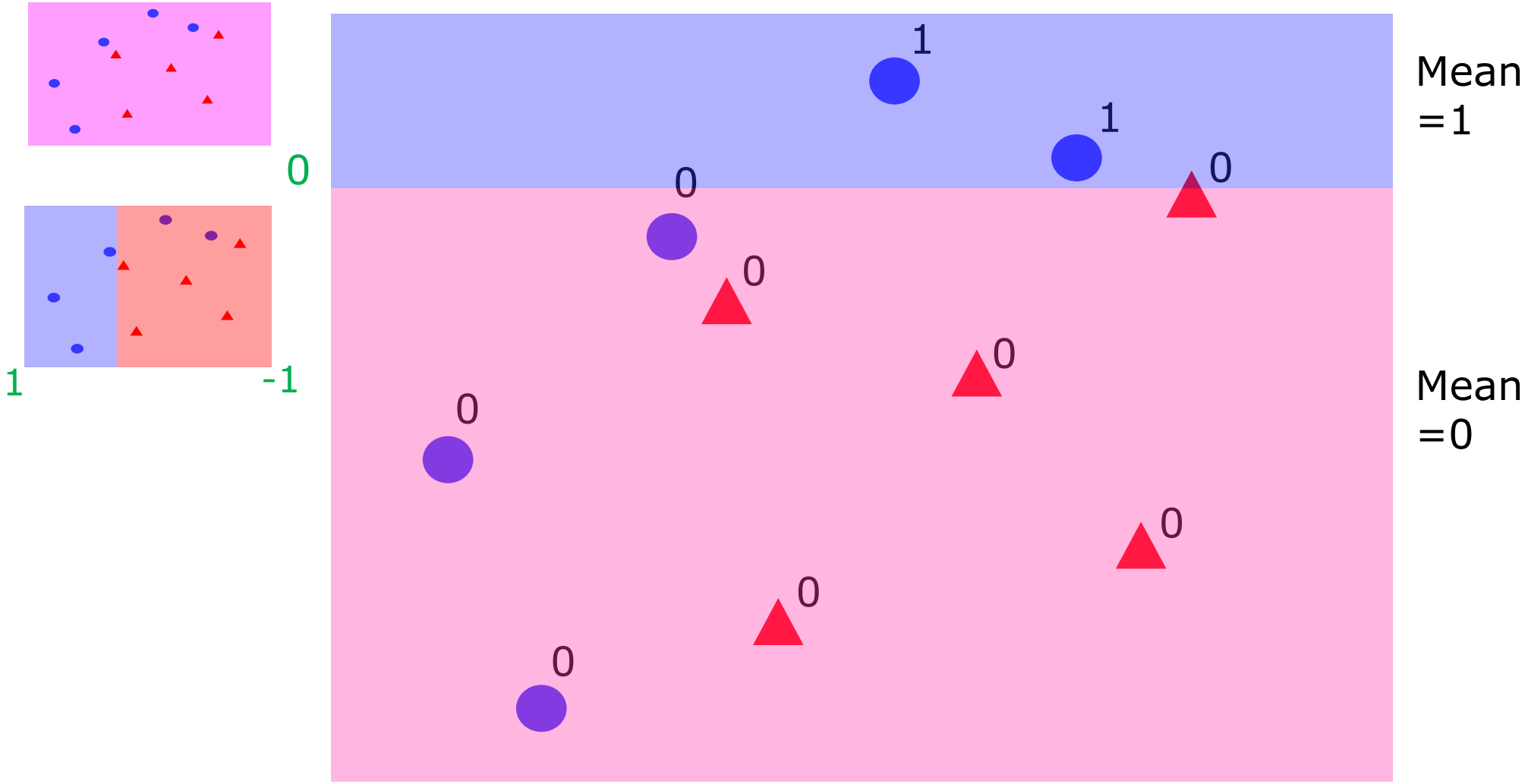
$\eta = 1$

$$\hat{y} = \begin{pmatrix} y_1 1_{1-y_1 F_{t-1}(x_1)>0} \\ \vdots \\ y_n 1_{1-y_n F_{t-1}(x_n)>0} \end{pmatrix}.$$

1 1 0 0 0 0 0 0 0 0 0



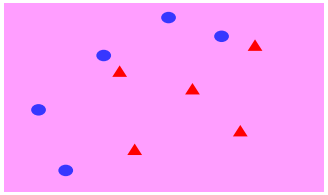
Gradient Boosting, Stumps, Hinge

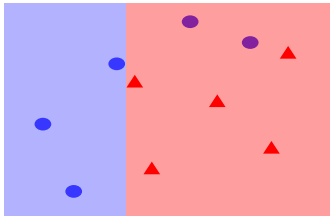


minimize

$\sum_{i \in l} \max(0, 1 - y_i(F_{t-1}(x_i) + c_l))$.

Gradient Boosting, Stumps, Hinge





1

-1

0

1

1

0

0

0

0

0

0

0

0

Mean =1

Mean =0

Derivative:

$\sum_{i \epsilon l} -y_i 1_{1-y_i(F_{t-1}(x_i)+c)>0} =$

$-2 \cdot 1_{1-(-1+c)>0}$.

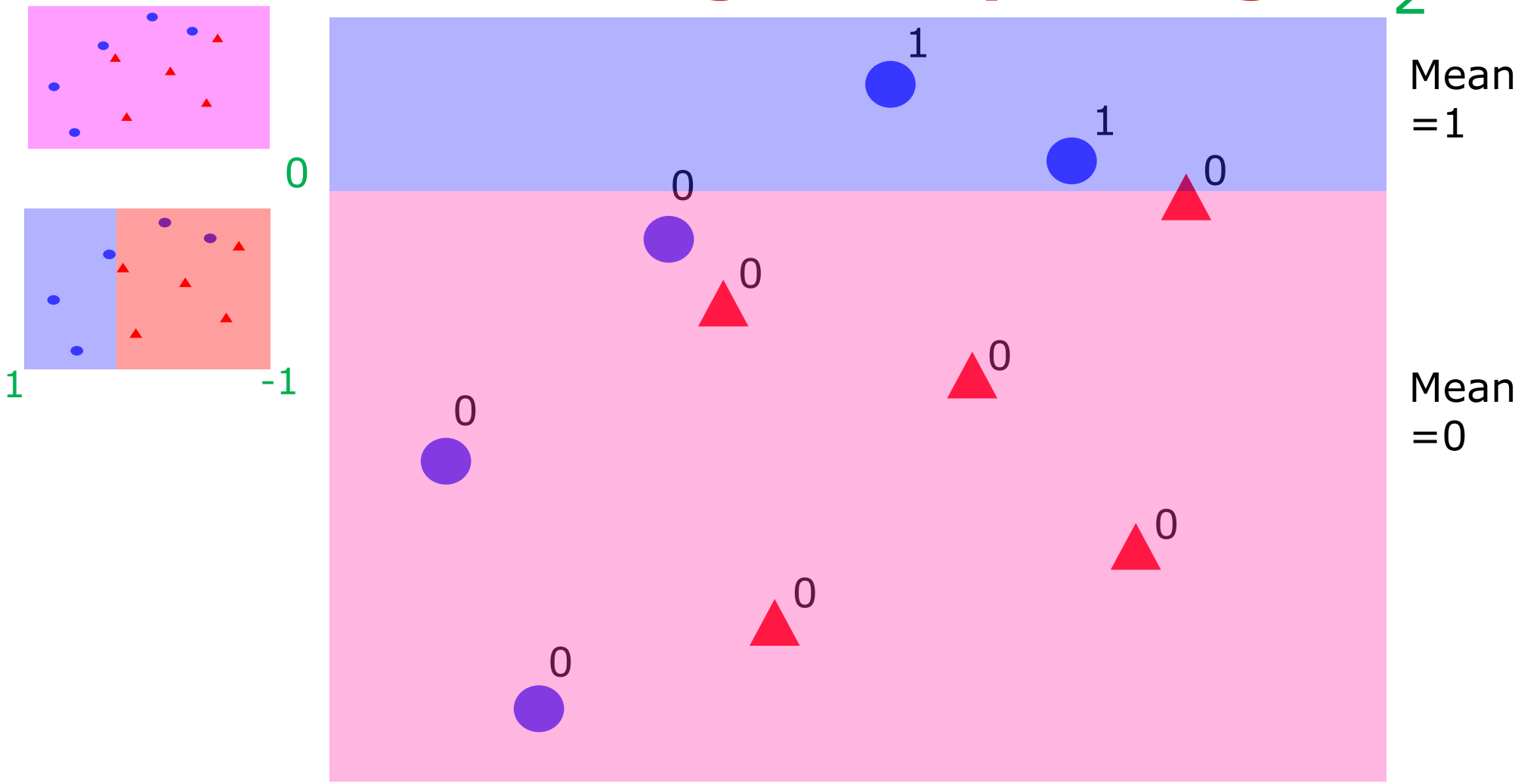
– For c<2.

0 for c≥2.

minimize

$\sum_{i \in l} \max(0, 1 - y_i(F_{t-1}(x_i) + c_l))$.

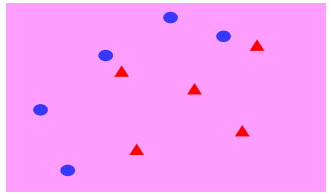
Gradient Boosting, Stumps, Hinge

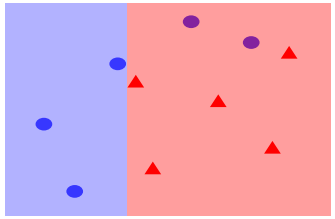


minimize

$\sum_{i \in l} \max(0, 1 - y_i(F_{t-1}(x_i) + c_l))$.

Gradient Boosting, Stumps, Hinge





1 -1

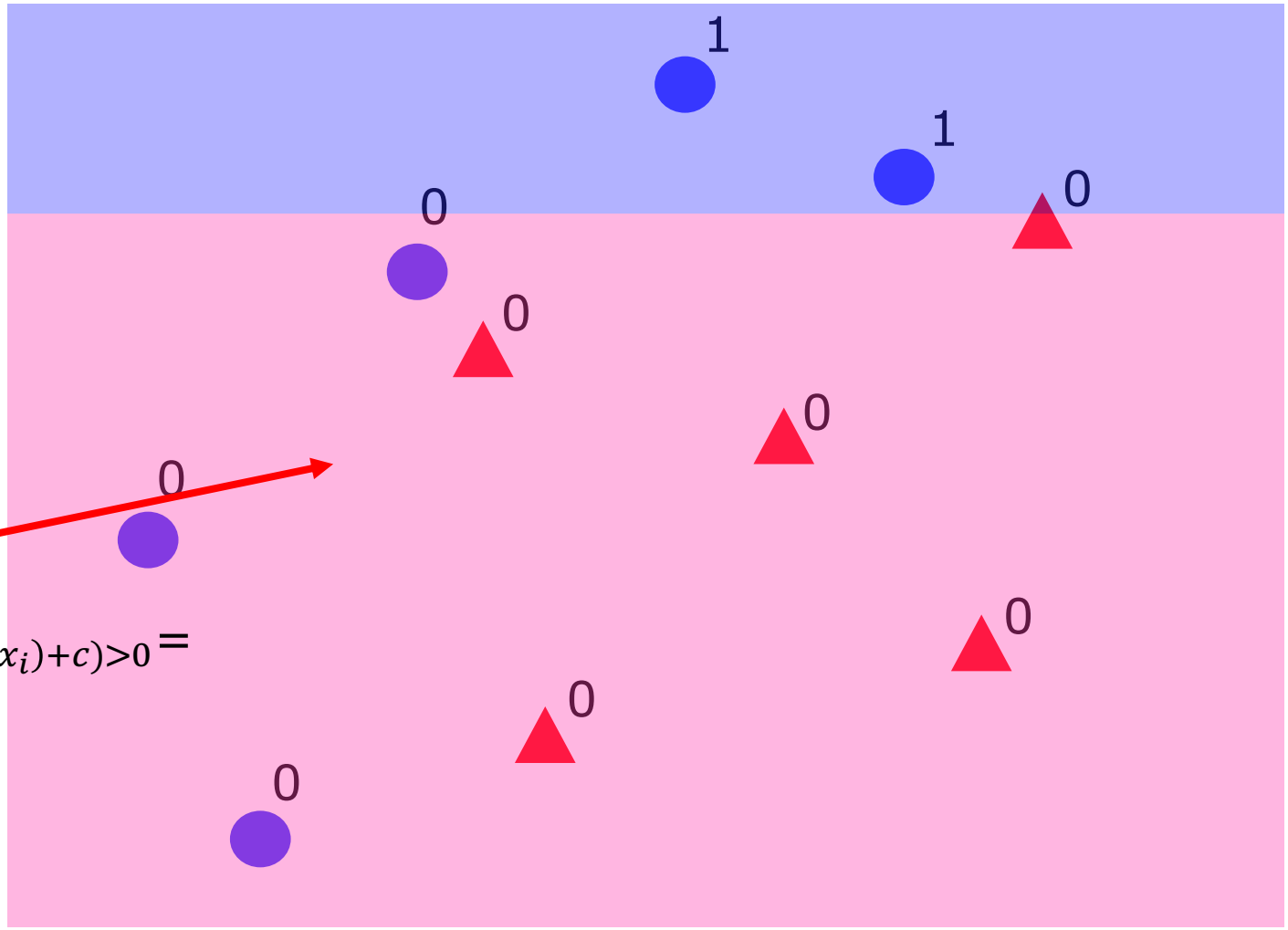
2

0

Mean =1

Mean =0

0



Derivative:

$\sum_{i \epsilon l} -y_i 1_{1-y_i(F_{t-1}(x_i)+c)>0} =$

$-3 \cdot 1_{1-(1+c)>0} +$

$5 \cdot 1_{1+(-1+c)>0}$.

– For c<0.

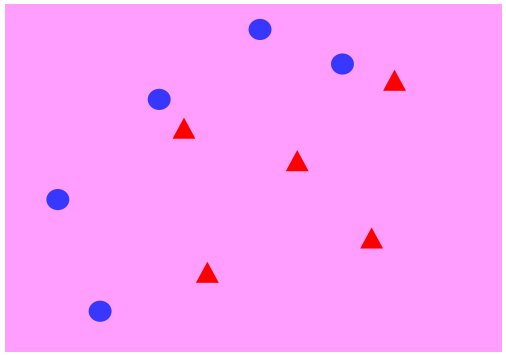
+ For c>0.

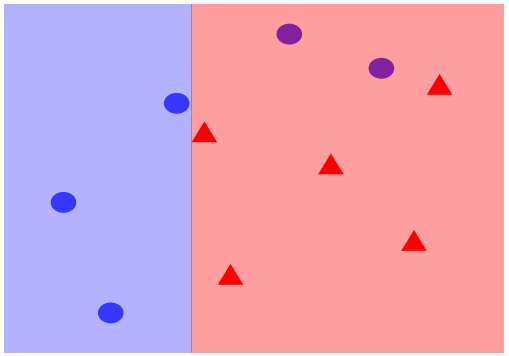
minimize

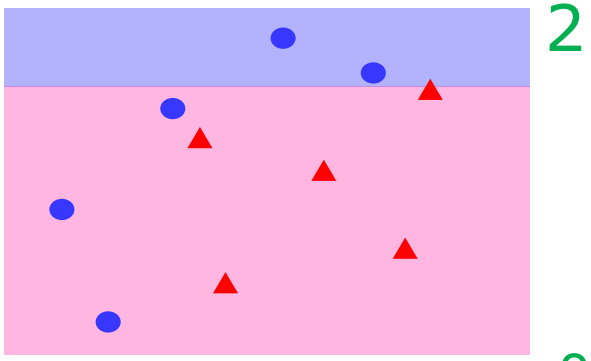
$\sum_{i \in l} \max(0, 1 - y_i(F_{t-1}(x_i) + c_l))$.



Gradient Boosting, Stumps, Hinge



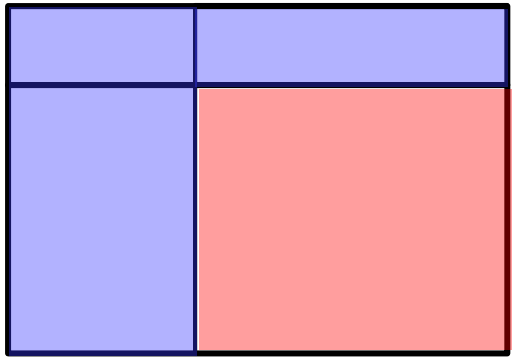




$$f(x) = h_1(x) + h_2(x) + h_3(x)$$

3	1
1	-1

$$\text{sign}(f(x))$$



Gradient Boosting

- Gradient Boosting:
  - Let $h_1(x) = c$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = -\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$.
    - For each leaf l in h_t:
      - Let l predict value c_l minimizing
        - $\sum_{i \in l} L(F_{t-1}(x_i) + c_l, y_i)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$



Gradient Boosting

- Gradient Boosting:
  - Let $h_1(x) = c$ where c minimizes $\mathrm{E}_{\text{in}} = \frac{1}{n}\sum_{i=1}^{n} L(c, y_i)$. Let $\alpha_1 = 1$.
  - For t=2,…,T:
    - Train regression tree h_t to minimize least squares loss with data X, $\hat{y} = -\begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$.
    - For each leaf l in h_t:
      - Let l predict value c_l minimizing
        - $\sum_{i \in l} L(F_{t-1}(x_i) + c_l, y_i)$
    - Let $\alpha_t = \eta$.
  - Output $f(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$

Already assumed!

If can compute derivative of $L(c,y_i)$ wrt. c, then can minimize using gradient descent in one variable!



Gradient Boosting

- Gradient Boosting:
  - General template for boosting with any pointwise loss function L(y',y).
  - Only need to be able to compute derivative wrt. y'.

- Can focus efforts on designing good algorithms for training regression trees with least squares loss.

- Why trees as base learners?
  - Leaves return constants.
  - To optimize for concrete loss function L, only need to be able to optimize constant in a leaf.
  - Other plus: Trees don't care about normalizing input features



Gradient Boosting

- Gradient Boosting:
  - One of the best off-the-shelf classifiers.

- Super efficient freely-available open-source implementations, such as XGBoost and LightGBM.

